Introduction to Computer Science

Overview of Computer Science

- Computer Science is not the study of computers, nor the practice of their use.
- Computer science is the study of information, protocols, and algorithms for idealized and real automata.
  - **Automaton:** “self-moving” – in our context, self “deciding” or autonomous mechanism with bounded resources (time and space)
  - **Information:** knowledge represented in a form suitable for transmission, manipulation, etc.
  - **Protocol:** rules for exchanging information without problems
  - **Algorithm:** an unambiguous, finite description in simple steps or actions

Cont…

❑**Science.**

- Systematic knowledge or practice.
- Acquiring knowledge through the scientific method of natural phenomena (natural sciences) or human or social behavior (social sciences).
- Like scientists, Computer Scientists observe the behavior of complex systems, form hypotheses, and test predictions.

Cont…

- Similar to mathematics, almost everyone in modern society uses computers.
- Getting a computer science degree prepares you for everything and nothing.
  - To do anything requires programming.
  - To do something useful requires domain knowledge.
- Do you become skillful at programming or an expert in a domain?

Cont…

- The single most important skill for a computer scientist is problem solving.
- It involves the ability to formulate problems, think creatively about solutions, and express solutions clearly and accurately.
- The process of learning to program is an excellent opportunity to develop problem solving skills.

Basic Concepts in Computer Science

- Computer science is a vast and multidisciplinary field that encompasses a wide range of concepts and principles including:
  - Computer Programming
  - Data Structures
  - Algorithms
  - Computational thinking
  - Software Engineering Principles.

Programming Fundamentals

- Computer programs, known as software, are instructions to the computer.
- You tell a computer what to do through programs.
- Without programs, a computer is an empty machine.
- Computers do not understand human languages, so you need to use computer languages to communicate with them.
- Programs are written using programming languages.

Cont…

- Examples of the most useful programming languages include:
  - Python
  - Java
  - C++
  - PhP
  - R
- Programming is the application of problem solving to creating executable computer programs.

Syntax and Semantics of a Programming Language

- The syntax and semantics of a programming language are two fundamental aspects that define how programs are written, interpreted, and executed.

❑Syntax:

- Syntax refers to the set of rules that dictate how programs written in a particular language should be structured.
- It defines the combinations of symbols, keywords, and operators that are considered to be correctly structured programs.

Syntax Characteristics

❑Grammar Rules:

- Programming languages have a formal grammar that outlines the rules for constructing valid statements and expressions.
- This includes rules for the arrangement of keywords, punctuation, and other elements.

❑Tokenization:

- The code is broken down into tokens, which are the smallest units in a program (e.g., keywords, operators, identifiers).
- Tokenization is the process of converting a program into these meaningful tokens.

Cont…

❑Hierarchy:

- Syntax often includes a hierarchy, defining the order in which different elements can be combined.
- For example, in many languages, a statement may consist of an expression, and an expression may consist of operands and operators.

❑Parsing:

- The process of parsing involves analyzing the code according to the syntax rules to create a parse tree or abstract syntax tree.
- This tree represents the grammatical structure of the program.

Cont…

❑Semantics:

- Semantics deals with the meaning of a program written in a programming language.
- It defines how the instructions of a program are executed or interpreted by a computer.
- In the larger world of linguistics, syntax is about the form of language, and semantics is about meaning.

Semantic Characteristics

❑Execution Rules:

- Semantics describe the behavior of each construct defined by the syntax during program execution.
- It answers questions like "What does this statement mean?" and "How is this statement executed?".

❑Operational Semantics:

- Describes the meaning of a program by specifying how its execution changes the state of the computation.
- It defines the effects of program statements on the storage, registers, and other components of the computing environment.

Cont…

- In summary, syntax defines the structure of a program.
- Semantics defines its meaning and behavior during execution.
- Both aspects are crucial for understanding and writing effective programs.

Cont…

- We want to specify syntax so that programmers can write valid programs.
- At the same time, compiler-writers can produce compilers that can cope with the full range of valid programs people produce.
- In this sense, constructs that can provide both the language definition and the mechanisms for processing programs within compilers are golden.

Variables, Data Types, and Expressions

❑Variables:

- A variable is a named storage location in a program that holds a value.
- It provides a way to label and store data in memory, allowing the program to manipulate and reference that data.
- Variables have names that follow specific naming conventions
  - e.g., no spaces, starts with a letter

Cont…

- Variables store data of a specific type (integer, floating-point, string, etc.).
- The data stored in a variable can be changed during the execution of the program.
- Variables are typically declared before use, specifying the data type and the name.
- The scope of a variable defines where in the program it can be accessed.
- Local variables are limited to a specific block or function, while global variables can be accessed throughout the program

Cont…

❑Data Types:

- Data types define the type of values that variables can hold.
- Common data types include integers, floating-point numbers, strings, booleans, arrays, and more.
- **Integer** - Represents whole numbers without any decimal points
- **Floating-point** - Represents numbers with decimal points.

Cont…

- **String** - Represents sequences of characters
- **Boolean** - Represents true or false values.
- **List/array** - Represents an ordered collection of items.
- **Character (or char) -** used for single letters.
- **Date** - stores a calendar date with other programming information

Cont…

❑Expressions:

- An expression is a combination of variables, constants, operators, and function calls that produces a value.
- Expressions are often used to perform calculations or create new values.
- Expressions use operators to perform operations on variables and values.
- Examples of operators include +, -, *, /, % (modulo), == (equality), and more.

Cont…

- Expressions are evaluated to produce a result.
- Example: result = x + y (the expression adds the values stored in variables x and y and assigns the result to the variable result).
- Expressions can be simple or complex, involving multiple operators and operands.
- Example: complex_expression = (a + b) * (c - d)

Control Structures

- Control structures are fundamental constructs in computer programming that determine the flow of a program's execution.
- They help control the order in which statements or instructions are executed.
- They allow you to make decisions, loop through tasks, and create more complex logic.
- The grouped into 3 groups:

Cont…

❑Sequential Control:

- Statements are executed one after the other in the order they appear in the program.
- This is the default control flow, where instructions are executed sequentially from top to bottom.

Selection Control (Conditional Statements):

- Conditional statements allow you to make decisions in your program based on a condition.
- The most common conditional statements are:

❑**If Statement:**

- Executes a block of code if a condition is true.
- General form:

```
If (condition){

        # do some action
}
```

Cont...

❑The If/else statement:

- While if only acts if the condition is true.
- If/else specifies an action to be performed both when the condition is true and when it is false.
- **General Form:**

```
if (x is true){
        # some action
}
else if (x not true){
      # some action
}
```

Cont…

- For example:

  If a student's grade is greater than or equal to 60:

  Print "Passed"

  Else:

  Print "Failed"

Cont…

❑If…else if…else statements:

- Test for multiple cases/conditions.
- Once a condition is met, the other statements are skipped.
- We add else at the end to handle the situation if no condition has been met.
- Basically, it is an advancement of if…else.

Cont…

- For Example:

If student's grade is greater than or equal to 90
Print "A"
Else If student's grade is greater than or equal to 80
Print "B"
else If student's grade is greater than or equal to 70
Print "C"
else If student's grade is greater than or equal to 60
Print "D"
else
Print "F"

Cont…

- General Format:

```
if (x and y is true){
        #some action
}
else if (x is true and y not true){
        #some action
}
else{
        #some action
}
```

Cont…

❑The switch statement:

A switch statement allows a variable to be tested for equality against a list of values.

Each value is called a case, and the variable being switched on is checked for each switch case.

Cont…

- The following rules apply to a switch statement:
  - You can have any number of case statements within a switch. Each case is followed by the value to be compared to and a colon.
  - The constant expression for a case must be the same data type as the variable in the switch
  - When the variable being switched on is equal to a case, the statements following that case will execute until a break statement is reached.
  - When a break statement is reached, the switch terminates, and the flow of control jumps to the next line following the switch statement.

Cont…

- Not every case needs to contain a break. If no break appears, the flow of control will fall through to subsequent cases until a break is reached.
- A switch statement can have an optional default case, which must appear at the end of the switch.
- The default case can be used for performing a task when none of the cases is true. No break is needed in the default case.

Cont…

- General format:

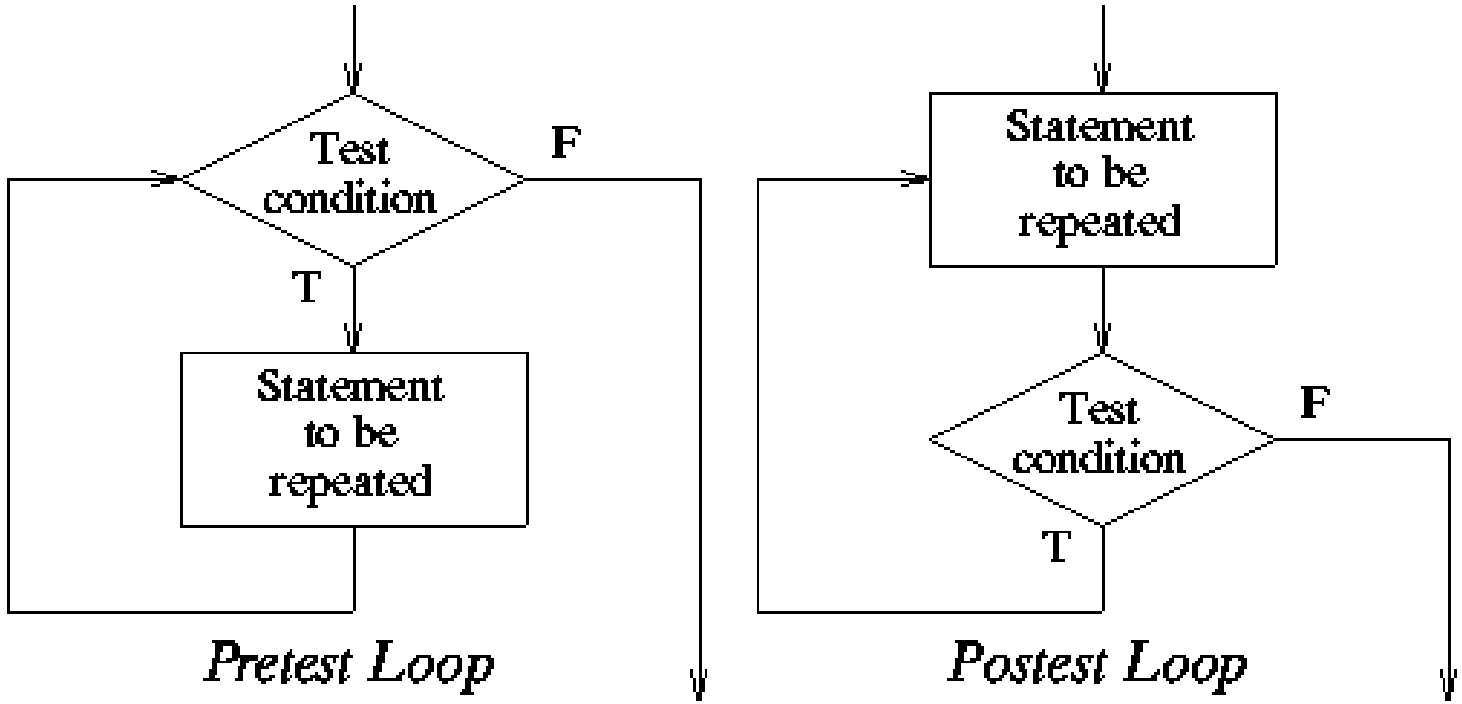
```
switch (action) {
    case 1:
        # some action
        break;
    case 2:
        # some action
        break;
default:
        # some action
}
```

REPETITION CONTROL STRUCTURES

- Also known as a loop.
- A loop statement allows the execution of a statement or a group of statements multiple times until a condition either tests true or false.
- There are two types of loops: Pre-test and post-test loops.
- In a pretest loop, a logical condition is checked before each repetition to determine if the loop should terminate. These loops include:
  - while loop
  - for loop

Cont…

- Post-test loops check a logical condition after each repetition for termination.
  - The do-while loop is a post-test loop.



WHILE LOOP IN C

- A while loop statement repeatedly executes a target statement as long as a given condition is true.
- The syntax of a while loop in C programming language is:

```
while(condition){
        #statement(s);
        #update expression
}
```

- The statement(s) may be a single statement or a block of statements. The loop iterates while the condition is true.
- When the condition becomes false, program control passes to the line immediately following the loop.

FOR LOOP

- A for loop is a repetition control structure that allows you to efficiently write a loop that needs to execute
- a specific number of times.
- The syntax of a for loop in C programming language is:

```
for ( initial expression; test expression/logical condition; update expression )
{
    statement(s);
}
```

Cont…

- Here is the flow of control in a for loop:
    1. This step initializes any loop control variables. You are not required to put a statement here, as long as a semicolon appears.
    2. Next, the condition is evaluated. If it is true, the body of the loop is executed. If it is false, the body of the loop does not execute and flow of control jumps to the next statement just after the for loop.
    3. After the body of the for loop executes, the flow of control jumps back up to the update expression. This statement allows you to update any loop control variables. This statement can be left blank, as long as a semicolon appears after the condition.
    4. The condition is now evaluated again. If it is true, the loop executes and the process repeats itself. After the condition becomes false, the for loop terminates.

DO...WHILE LOOP

- Unlike the for and while loops, which test the loop condition at the top of the loop, the do...while loop checks its condition at the bottom of the loop.
- A do...while loop is similar to a while loop, except that a do...while loop is guaranteed to execute at least one time.
- The structure loops until a condition tests false, i.e., loop until.

Cont…

❑**Syntax**

```
do
{
      # statement(s)

}while( condition )
```

- If the condition is true, the flow of control jumps back up to do, and the statement(s) in the loop execute again.
- This process repeats until the given condition becomes false.

Functions and Procedures

❑A Function:

- The function is one of the fundamental thoughts in computer programming.
- It is used to calculate something from a given input.
- Hence it got its name from Mathematics.
- The function can be either user-defined or predefined.
- The function program has a block of code that performs some specific tasks or functions.

Cont…

- They can accept parameters, perform calculations, manipulate data, and return results.
- Depending on the programming language you use, you can also compile a library of functions and import them for use in other programs.

Cont…

- In python, the function general form is:

```
def myFunction(argurments):

    #some action

    return value(s)
```

Cont…

❑Procedures:

- Procedural programming is a programming paradigm, derived from imperative programming, based on the concept of the procedure call.
- Procedures is a programming component that contains a series of computational steps to be carried out.
- Any given procedure might be called at any point during a program's execution, including by other procedures or itself.

Cont…

- Functions and procedures can offer several advantages in programming, such as reusability, modularity, security, and performance.
  - **Reusability** allows you to write a function or procedure once and use it in multiple places avoiding duplication and inconsistency.
  - **Modularity** enables you to break down complex tasks into smaller and simpler subprograms, making your code easier to read, understand, and debug.
  - **Security** enables you to grant or revoke permissions to execute specific functions or procedures, limiting the access and actions of different users or roles.

Data Structures and Algorithms

Introduction

- A computer program is a collection of instructions to perform a specific task.
- For this, a computer program may need to store data, retrieve data, and perform computations on the data.
- A data structure is a named location that can be used to store and organize data.
- An algorithm is a collection of steps to solve a particular problem.
- Learning data structures and algorithms allow us to write efficient and optimized computer programs.

What is an Algorithm?

- In computer programming terms, an algorithm is a set of well-defined instructions to solve a particular problem.
- It takes a set of input(s) and produces the desired output. For example,
- An algorithm to add two numbers:
  - *Take two number inputs*
  - *Add numbers using the + operator*
  - *Display the result*

Cont…

❑Qualities of a Good Algorithm

- Input and output should be defined precisely.
- Each step in the algorithm should be unambiguous.
- Algorithms should be most effective among many different ways to solve a problem.
- An algorithm shouldn't include computer code.
- Instead, the algorithm should be written in such a way that it can be used in different programming languages.

Data Structure and Types

❑What are Data Structures?

- Data structure is a storage that is used to store and organize data.
- It is a way of arranging data on a computer so that it can be accessed and updated efficiently.
- **Note:** Data structure and data types are slightly different.
- Data structure is the collection of data types arranged in a specific order.

Cont…

❑Types of Data Structure

- Basically, data structures are divided into two categories:
    1. Linear data structure
    2. Non-linear data structure
- Let's learn about each type in detail.

Linear data structures

- In linear data structures, the elements are arranged in sequence one after the other.
- Since elements are arranged in a particular order, they are easy to implement.
- However, when the complexity of the program increases, the linear data structures might not be the best choice because of operational complexities.

Examples:

❑Array Data Structure

- In an array, elements in memory are arranged in continuous memory. All the elements of an array are of the same type.
- And, the type of elements that can be stored in the form of arrays is determined by the programming language.

- **Stack Data Structure**
  - In stack data structure, elements are stored in the LIFO principle.
  - That is, the last element stored in a stack will be removed first.
  - It works just like a pile of plates where the last plate kept on the pile will be removed first.

Cont…

❑Linked Lists:

- Linked lists are composed of nodes, each containing a data element and a reference to the next node.
- They are efficient for insertion and deletion but have slower random access compared to arrays.

❑Queues:

- Queues follow the First-In, First-Out (FIFO) principle.
- Elements are enqueued at the rear and dequeued from the front.
- Queues are used for tasks like managing processes and scheduling.

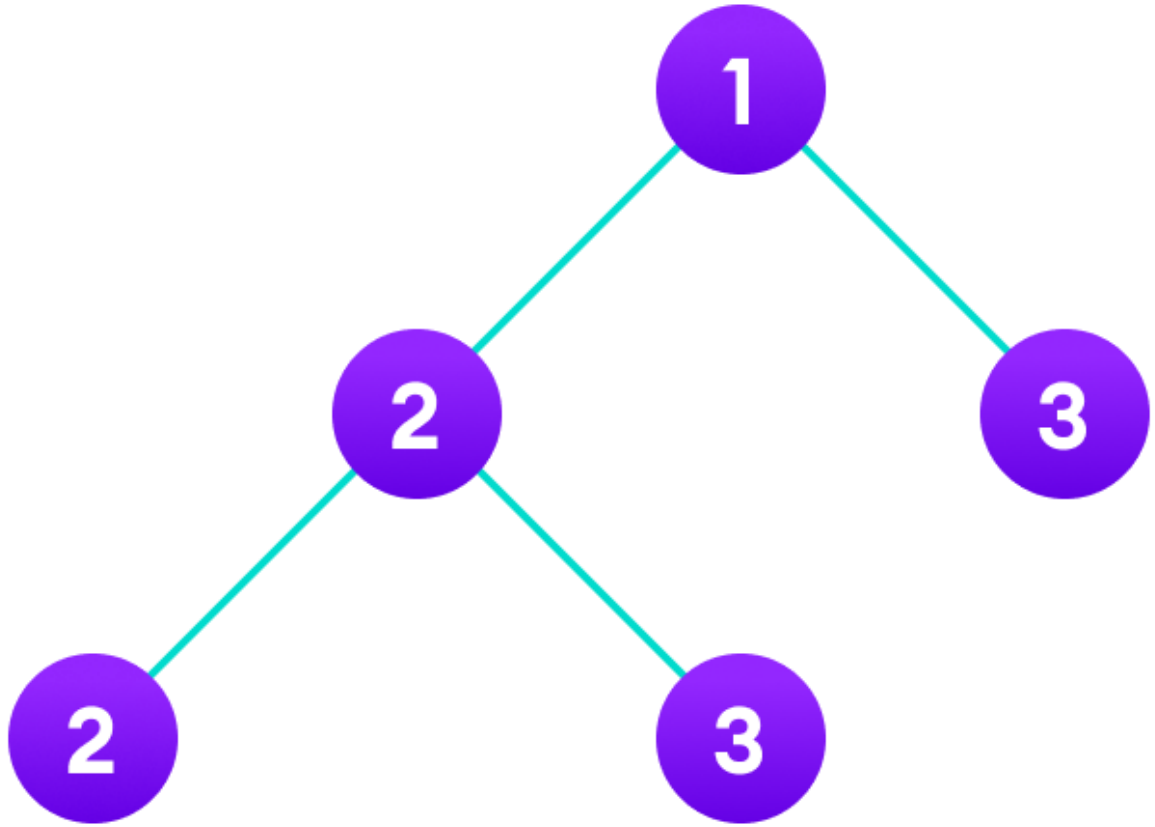
Non-linear data structures

- Unlike linear data structures, elements in non-linear data structures are not in any sequence.
- Instead they are arranged in a hierarchical manner where one element will be connected to one or more elements.
- Non-linear data structures are further divided into graph and tree based data structures.

Cont...

❑ **Trees**

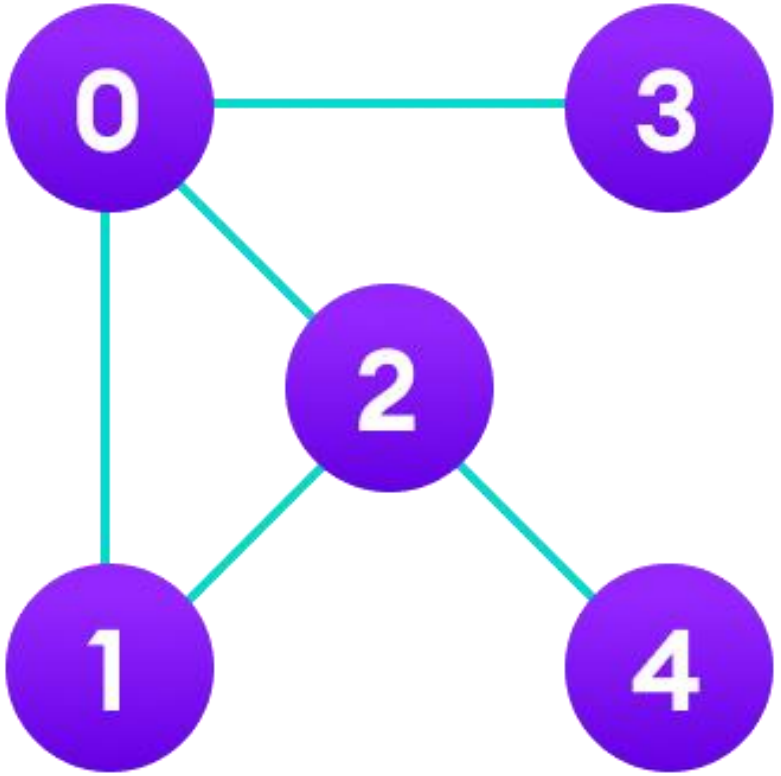
- Trees are hierarchical data structures with a root node and child nodes.
- Common types of trees include binary trees, binary search trees (BST), and AVL trees.
- In graph data structure, each node is called vertex and each vertex is connected to other vertices through edges.



Cont...

❑ **Graphs:**

- Graphs consist of nodes (vertices) and edges connecting them.
- They are used to represent complex relationships between objects and are essential for tasks like network analysis and mapping.
- Similar to a graph, a tree is also a collection of vertices and edges.
- However, in tree data structure, there can only be one edge between two vertices.



Sorting and Searching Algorithms

- Sorting and searching algorithms are fundamental techniques in computer science.
- They are used in a wide range of applications.
- Sorting algorithms arrange a list of elements in a specific order.
- Searching algorithms are used to find a particular element within a given list.

Sorting Algorithms.

- Sorting algorithms are classified into the following groups:

❑**Bubble Sort:**

- Bubble sort repeatedly steps through the list, compares adjacent elements, and swaps them if they are in the wrong order.
- This process continues until the entire list is sorted. I
- t's not very efficient and is mainly used for educational purposes.

Cont…

❑Selection Sort:

- Selection sort divides the input list into two parts: the sorted part and the unsorted part.
- It repeatedly selects the smallest element from the unsorted part and moves it to the sorted part.

❑Insertion Sort:

- Insertion sort builds the final sorted array one item at a time.
- It takes each element and inserts it into its correct position in the already sorted part of the list.

Cont…

❑Merge Sort:

- Merge sort is a divide-and-conquer algorithm that divides the input list into smaller sublists, sorts them, and then merges the sorted sublists.

❑Quick Sort:

- Quick sort is another divide-and-conquer algorithm.
- It selects a "pivot" element and partitions the list into two sublists, one containing elements less than the pivot and one containing elements greater than the pivot.
- It recursively sorts the sublists.

Searching Algorithms:

❑Linear Search:

- Linear search is a simple search algorithm that sequentially checks each element in the list until a match is found or the whole list has been searched.

❑Binary Search:

- Binary search is an efficient search algorithm for sorted lists.
- It repeatedly divides the search interval in half until the target element is found or the search interval is empty.

Cont…

❑Hashing:

- Hashing is a technique that uses a hash function to map keys to locations in a data structure called a hash table.
- It allows for fast retrieval of values based on their keys.

❑Interpolation Search:

- Interpolation search is an improved search algorithm for sorted lists that uses interpolation to estimate the position of the target element.
- It determines the target element's position based on its value and the extreme values within the currently searched section of the array.

Cont…

❑Exponential Search:

- Exponential search is used for unbounded lists.
- It involves finding a range that potentially contains the target element using exponential steps and then performing a binary search within that range.